



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Symbiosis International (Deemed University)

(Established under section 3 of the UGC Act, 1956)

Re-accredited by NAAC with 'A' grade (3.58/4) | Awarded Category – I by UGC

Founder: Prof. Dr. S. B. Mujumdar, M. Sc., Ph. D. (Awarded Padma Bhushan and Padma Shri by President of India)

Assignment No. 10

Subject:	DevOps Lab
TCODE:	TE7950
Name of Student	Adarsh Jha
PRN No.	23070122261
Branch	CSE, Batch (2023-27), L4
Academic Year & Semester	25 – 26, Sem VII
Date of Performance	23 rd July, 2026
Title of Assignment:	Docker Volumes & Data Persistence
Theory:	Docker volumes provide persistent storage for containers by storing data outside the container's writable layer. Unlike container storage, Docker volumes remain intact even after a container is stopped or removed, allowing data to be shared across multiple container instances. By mounting a named volume to a directory inside the container, applications can read and write data that persists independently of the container lifecycle. This makes Docker volumes ideal for storing application data, logs, databases, and other important files that should not be lost when containers are recreated. Commands Used • Built the updated Docker image with a new version tag:. • Created a Docker named volume: • Started a container with the named volume mounted: • Stopped the running container: • Removed the stopped container: • Started a new container using the same Docker volume:

Implementation:

Step 1: Modify the Python Application

Modified Flask Application — Updated the Flask application to write timestamped greetings to a log file inside the /app/data directory and configured the application to run on port

Step 2: Update the Dockerfile

Updated Dockerfile — Modified the Dockerfile to expose `8080`, matching the updated Flask application configuration.

Step 3: Build the New Docker Image

Built Updated Docker Image — Built a new Docker image named `my-flask-app:v2` containing the modified Flask application configured for Docker volume-based data persistence.

Step 4: Create a Docker Named Volume

Created Docker Volume — Created a Docker named volume called my-app-data to store application data independently of the container, ensuring data persistence across container restarts and removal.

Step 5: Access the Application and Write Data

Verified Data Logging — Accessed the Flask application and refreshed the page multiple times to verify that timestamped greetings were successfully written to the persistent log file stored in the Docker volume.

Step 6: Stop and Remove the Container

Stopped and Removed Container — Stopped the flask-volume-demo container and removed it using the docker stop and docker rm commands while preserving the application data stored in the Docker named volume.

Step 7: Verify Data Persistence

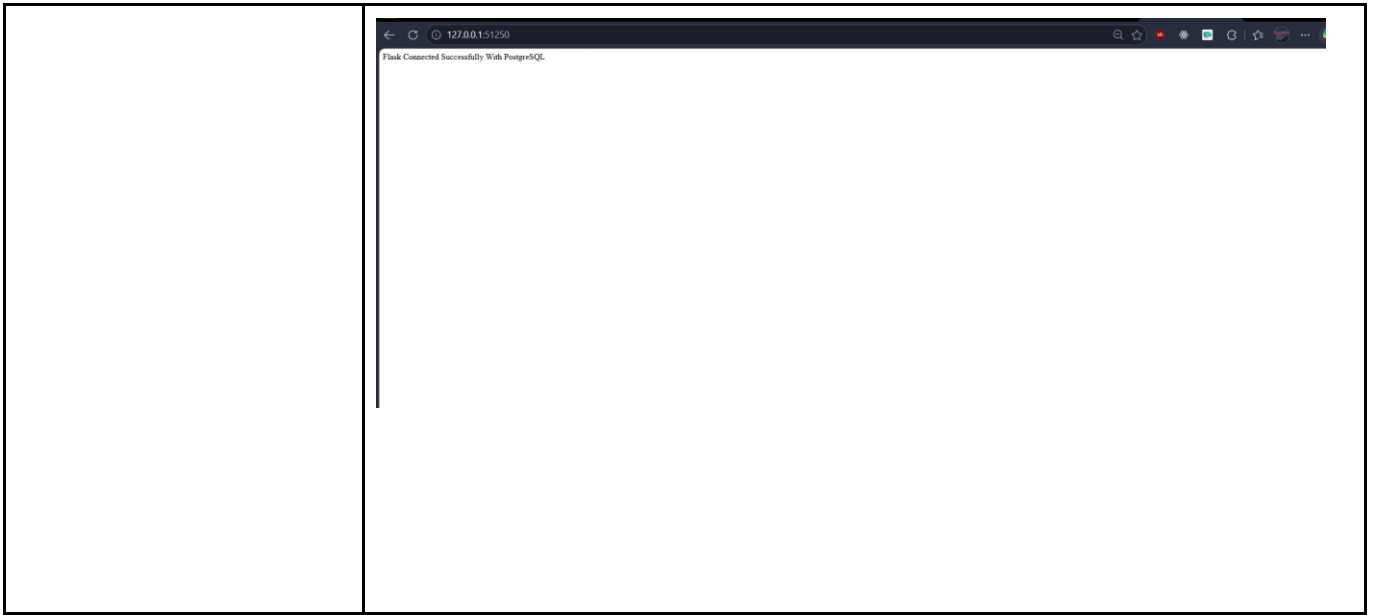
Verified Data Persistence — Accessed the Flask application after starting a new container with the same Docker named volume and confirmed that the previously stored log entries were retained. This demonstrated successful data persistence across multiple container instances.

```

ps C:\WINDOWS\system32 docker images

IMAGE                                ID                                DISK USAGE  CONTENT SIZE  U In Use
ad4r5h/flask-app:latest             a829a95381148                    1.62GB      417MB
alpine:latest                        28bdc5fe8b56d                    1.13MB      3.93MB
assignment-t2-advanced-devops-flask-app:latest 7109d2345dc8                    1.62GB      417MB
docker.n8n.io/n8nio/n8n:latest       cda6bafc7bb04                    2.47GB      387MB
gcr.io/k8s-minikube/kicbase:v0.9.56  b9707f5e09a9e                    1.94GB      545MB
gcr.io/k8s-minikube/kicbase256:eb4fec080ad7badf8e6436195cc29252f5fb859f5acfd5bd8d9deb576f3e93 bbfce90e8ad                    1.94GB      545MB
jenkins-agent-maven:latest           83fcae8abfb99                    821MB      248MB
jenkins/inbound-agent:latest         677cfa49f8642                    479MB      151MB
jenkins/jenkins-lts                  f4656e6cc1c08                    893MB      351MB
jenkins/2.60.3                       eeb4850eb5f5f                    1.88GB      316MB
my-flask-app:latest                  bd96368293d00                    200MB      48.8MB
n8nio/n8n:latest                     cda6bafc7bb04                    2.47GB      387MB
postgres:15                          74e11bc41804                      633MB      164MB
react-docker-app:latest              8b566310983                      547MB      138MB
ps C:\WINDOWS\system32 docker rmi ad4r5h/flask-app:latest
Untagged: ad4r5h/flask-app:latest
Deleted: sha256:a29a953148bba860f88bc0fde5401bf78a74087342c8955cb518eac54f6f
ps C:\WINDOWS\system32 docker pull ad4r5h/flask-app:latest
latest: Pulling from ad4r5h/flask-app
077199981d22: Pull complete
263f6dc26961: Pull complete
8175c191dde6: Pull complete
efc0dbd4f07: Download complete
Digest: sha256:a29a953148bba860f88bc0fde5401bf78a74087342c8955cb518eac54f6f
Status: Downloaded newer image for ad4r5h/flask-app:latest
docker.io/ad4r5h/flask-app:latest
ps C:\WINDOWS\system32 docker images

```



Subject Faculty Name: Prof. Arijit Dutta